

Introduction à l'algorithmique et à la
programmation

Les procédures et fonctions en C

Les procédures et fonctions en C

Buts :

- **ré-utilisabilité** : aptitude d'un algorithme à être réutilisé pour résoudre des tâches équivalentes à celle pour laquelle il a été conçu
- simplification : en allégeant le code

Vous en utilisez déjà ...

```
printf("la valeur de a est %d", a);  
scanf("%d",&a);  
strcpy(ch1, "bonjour");  
strcat(ch_destination, ch_a_ajouter);  
int main()
```

Les procédures et fonctions en C

Plan :

- O Les paramètres formels
- O Les paramètres effectifs
- O les fonctions
- O les spécificités du C

Les procédures et fonctions en C

Les paramètres formels

O En C il n'y a que deux modes de transmission :

- en entrée
- en entrée/sortie

O Déclaration

- en entrée

entF <nom>: type **se traduit par** <type> <nom>

- en entrée/sortie

entF/sortF <nom>: type **se traduit par** <type> *<adr_nom>

- le paramètre formel sera une adresse (... la ou est rangée la variable ...)
- L'opérateur * veut dire
 - « la valeur rangée à l'adresse ... »
 - « le contenu de ... »
- Bonne pratique :
 - le nom du paramètre formel commence par **adr_**

Les procédures et fonctions en C

Exemple 1 : Paramètres d'entrée uniquement :

La procédure calcule et affiche la moyenne de deux entiers

O En pseudo-code

```
procedure affiche_moyenne(entF v1: entier, entF v2: entier) c'est  
  debut  
    moy : float  
    moy := (v1+v2)/2.0;  
    afficheEcran(" la moyenne de", v1, "et de ", v2, " est :", moy)  
  fin
```

O En langage C

```
void affiche_moyenne(int v1, int v2){  
  float moy ; // déclaration d'une variable locale pour le calcul  
  moy = (v1 + v2)/2.0;  
  printf(" la moyenne de %d et de %d est %f", v1, v2, moy);  
}
```

Remarque : il faut ajouter "void" pour préciser que la procédure ne retourne pas de valeur

Les procédures et fonctions en C

Exemple 2 : Paramètres d'entrée et de **sortie** :

La procédure calcule la moyenne de deux entiers

O En pseudo-code

```
procedure moyenne(entF v1: entier, entF v2: entier, sortF moy : reel) c'est  
  debut  
    moy := (v1+v2)/2.0;  
  fin
```

O En langage C

```
void moyenne(int v1, int v2, float *adr_moy){  
  *adr_moy = (v1 + v2)/2.0;  
}
```

Remarque : le paramètre formel est une adresse !!!

C'est bien *de contenu de la variable* adr_moy qui est de type float

Pour modifier la moyenne, il faut modifier le *contenu de* adr_moy

Les procédures et fonctions en C

Plan :

- O Les paramètres formels
- O Les paramètres effectifs
- O les fonctions
- O les spécificités du C

Les procédures et fonctions en C

Les paramètres effectifs

- O Les paramètres effectifs en entrée sont peuvent être
 - une variable
 - une constante
 - une valeur
- O Si le paramètre effectif est en entrée/sortie il faut passer l'adresse du paramètre.

& <variable> est l'adresse de la variable

Exemple :

```
int main(){  
    int nb1, nb2;  
    printf(" entrer deux entiers : ") ;  
    scanf("%d %d",&nb1,&nb2); (1)  
    return 0;  
}
```

(1) nb1 et nb2 sont des paramètres de sortie (la fonction scanf va modifier les variables nb1 et nb2) => il faut fournir l'adresse de nb1 et de nb2

Les procédures et fonctions en C

Déclaration des procédures en C

O Deux Possibilités :

- L'écriture de la procédure se fait avant le main
- L'écriture de la procédure se fait après le main, auquel cas il faut déclarer le **prototype** de la procédure avant le main.

Exemple : calcul de la moyenne de deux entiers

// le prototype est une instruction qui décrit l'utilisation de la procédure

void moyenne(**int** v1, **int** v2, **float** ***adr_moy**); // ; indispensable en fin de ligne

int main(){

int nb1, nb2;

float moyenne;

 printf(" entrer deux entiers : ") ;

 scanf("%d %d",&nb1,&nb2);

 moyenne(nb1, nb2, &moyenne) ; // Appel de la procedure

 printf("la moyenne de%d et%d est%f", nb1, nb2, moyenne) ;

return 0 ;

}

// définition de la procédure

...

Les procédures et fonctions en C

Exemple complet : calcul de la moyenne de 2 entiers

```
void moyenne(int v1, int v2, float *adr_moy) ;
```

```
int main(){  
    int nb1, nb2;  
    float moyenne;  
    printf(" entrer deux entiers : ") ;  
    scanf("%d %d",&nb1,&nb2);  
    moyenne(nb1, nb2, &moyenne) ; // Appel de la procédure  
    printf("la moyenne de%d et%d est%f", nb1, nb2, moyenne) ;  
    return 0 ;  
}
```

```
// définition de la procedure  
void moyenne(int v1, int v2, float *adr_moy){  
    *adr_moy = (v1 + v2)/2.0;  
}
```

Les procédures et fonctions en C

Plan :

- O Les paramètres formels
- O Les paramètres effectifs
- O les fonctions
- O les spécificités du C

Les procédures et fonctions en C

En C : pas de distinction entre procédures et fonctions

- O Une procédure est une fonction qui retourne un résultat vide (i.e. de type void)
- O Les fonction :

```
<type de résultat> <nom de la fonction> (<liste et type des paramètres>) {  
    <suite des instructions>  
    return <variable de bon type>;  
}
```

- O l'instruction return
 - met fin à l'exécution de la fonction
 - renvoie la valeur de la variable vers le programme appelant
- O **Bonne pratique** : une variable locale spécifique pour le retour

```
<type de résultat> <nom de la fonction> (<liste et type des paramètres>) {  
    <type de résultat> <resultat> ;  
    <suite des instructions>  
    return <resultat>;  
}
```

Les procédures et fonctions en C

Exemple de déclaration d'une fonction

la fonction calcule et retourne la moyenne de deux entiers

O En pseudo-code

```
fonction moyenne(entF v1: entier, entF v2: entier) délivre réel c'est  
  debut  
    moy : reel;  
    moy:= (v1+v2)/2.0;  
    retourne moy;  
  fin
```

O En langage C

```
float moyenne(int v1, int v2){  
  float moy ;  
  moy = (v1+v2)/2.0;  
  return moy  
}
```

Les procédures et fonctions en C

Les fonctions en C : Exemple complet du calcul de la moyenne de 2 entiers

```
// prototype de la fonction
```

```
float moyenne(int v1, int v2);
```

```
// Définition du programme principal
```

```
int main(){
```

```
    int a, b;
```

```
    float moy;
```

```
    moy = moyenne(11,12); // appel de la fonction
```

```
    printf("Voici la moyenne : %f ",moy);
```

```
    a=20;
```

```
    moy = moyenne(11,a); // appel de la fonction
```

```
    printf("Voici la moyenne : %f ", moy);
```

```
    moy = moyenne(a,a); // appel de la fonction
```

```
    printf("Voici la moyenne : %4.2f ", moy);
```

```
    return EXIT_SUCCESS;
```

```
}
```

```
// Définition de la fonction
```

```
float moyenne(int v1, int v2){
```

```
    float resultat ;
```

```
    resultat = (v1+v2)/2.0;
```

```
    return resultat ;
```

```
}
```

Les procédures et fonctions en C

Autre exemple : Toto et Titi échangent leur notes

```
void echange(int *adr_n1, int *adr_n2);  
int main(){  
    int n_toto, n_titi;  
    printf("Note de Toto");  
    scanf("%d",&n_toto);  
    printf("Note de Titi");  
    scanf("%d",&n_titi);  
    echange(&n_toto,&n_titi);  
    printf("Note de Toto : %d, note de Titi : %d",n_toto,n_titi);  
    printf("Note temporaire : %d",tmp); //n'est pas connu ici !  
    return EXIT_SUCCESS;  
}  
void echange(int *adr_n1, int *adr_n2){  
    int tmp;    // déclaration d'une variable locale à la procédure  
    tmp=*adr_n1;    // sauvegarde du contenu de adr_n1  
    *adr_n1=*adr_n2; // affectation du contenu de adr_n1 par le contenu de adr_n2  
    *adr_n2=tmp;    // affectation du contenu de adr_n2 par la valeur de tmp  
}
```

Les procédures et fonctions en C

Exemple de récursivité : factorielle

```
// prototype de la fonction
```

```
long int factorielle(int a); // type de retour long int car la factorielle peut est une valeur importante
```

```
int main(){
```

```
    int nb,
```

```
    long int resultat;
```

```
    printf("Entrer un nombre entier :");
```

```
    scanf("%d",&nb);
```

```
    resultat = factorielle(nb)
```

```
    printf("%d != %d", nb, resultat );
```

```
    return EXIT_SUCCES;
```

```
}
```

```
long int factorielle(int a){
```

```
    long int res;
```

```
    if (a==0)
```

```
        res=1;
```

```
    else
```

```
        res=a*factorielle(a-1); // appel recursif
```

```
    return res;
```

```
}
```


Les procédures et fonctions en C

Plan :

- O Les paramètres formels
- O Les paramètres effectifs
- O les fonctions
- O les spécificités du C
 - Les chaînes de caractères en paramètres
 - Le main, une fonction spécifique

Les procédures et fonctions en C

Les chaînes de caractères sont toujours en entrée/sortie une variable chaîne de caractères est déjà un adresse ...

```
void mariage(char *nomH, char *nomF);
```

```
int main(){
```

```
    char nomHomme[50];
```

```
    char nomFemme[50];
```

```
    printf("Nom de la femme : ");
```

```
    scanf("%s",nomFemme);                // nomFemme est une adresse
```

```
    printf("Nom de l'homme : ");
```

```
    scanf("%s",nomHomme);                // nomHomme est une adresse
```

```
    mariage(nomHomme,nomFemme); // appel de la fonction => 2 paramètres d'entrée/sortie
```

```
    printf("nom femme : %s nom homme : %s\n",nomFemme, nomHomme);
```

```
    return EXIT_SUCCESS;
```

```
}
```

```
void mariage(char *nomH, char *nomF){
```

```
    // Le paramètre formel est une adresse, c'est le contenu qui est le premier caractère de la chaîne
```

```
    char nom[50];        // variable locale pour définir le nouveau nom
```

```
    strcpy(nom, nomF);
```

```
    strcat(nom, "-");
```

```
    strcat(nom, nomH); // nom devient nomF-nomH
```

```
    strcpy(nomF,nom);  // mise à jour des noms
```

```
    strcpy(nomH,nom);
```

```
}
```

Les procédures et fonctions en C

Un main avec des paramètres ...

```
int main(int argc, char **argv){  
    // argc est le nombre de paramètres effectivement  
    // argv est un tableau de chaîne de caractères comprenant tous les paramètres  
    // argv[0] est le nom de la programme  
    // argv[1] : premier argument passé au programme  
    // ...  
    // argv[n] est le n-ième argument passé au programme  
    printf("nombre de paramètres : %d\n", argc);  
    // affichage de l'ensemble des paramètres du main  
    int i;  
    for (i=0; i<argc; i++){  
        printf("paramètre %d : %s\n",i, argv[i]);  
    }  
    return 0;  
}
```

```
$ gcc parametres_main.c -Wall -o parametres_main.out  
$ ./parametres_main.out abc 123 toto  
nombre de paramètres : 4  
paramètre 0 : ./parametres_main.out  
paramètre 1 : abc  
paramètre 2 : 123
```

Les procédures et fonctions en C

paramètre 3 : toto